

Challenge 3.

A matrix-free parallel solver for the Laplace equation

Luca Formaggia, Carlo de Falco, Paolo Joseph Baioni, Beatrice Crippa

AA 25-26

1 Introduction

Consider the Laplace equation

$$\left\{ \begin{array}{ll} -\Delta u = f(x), & \text{in } \Omega = (0, 1)^2, \\ u = 0, & \text{on } \{x = 0\}, \\ u = 0, & \text{on } \{x = 1\}, \\ u = 0, & \text{on } \{y = 0\}, \\ u = 0, & \text{on } \{y = 1\}, \end{array} \right. \quad (1)$$

which models heat diffusion over a square domain with a prescribed temperature (Dirichlet conditions) on the whole boundary.

A possible approach to solving this problem is the so-called *Jacobi iteration* method: given a uniform Cartesian decomposition of Ω consisting of n points along each coordinate direction, the goal is to find the discrete solution $u_{ij} = u(x_i, y_j)$, $i, j = 1, \dots, n$ at each point of this Cartesian grid.

We aim to represent the solution as a dense matrix U of size $n \times n$; the matrix is initialized with zeroes, except for the first and last rows and columns, which contain the boundary-condition values defined in eq. (1).

The scalar algorithm consists of the following iterative procedure.

Set $U^{(0)}(i, j) = 0, \forall i, j = 2, \dots, n-1$, and set the boundary nodes to the Dirichlet values. In this simple case

$$U^{(0)}(i, j) = 0, i = 1, \dots, n, j \in \{0, n\}, \text{ and } j = 1, \dots, n, i \in \{0, n\}.$$

In more general, non-homogeneous cases, however, the values may be different from 0.

For $k = 0, \dots$ until convergence:

1. update each *internal* entry as the average of the values of a four-point stencil

$$U^{(k+1)}(i, j) = \frac{1}{4h^2} \left(U^{(k)}(i-1, j) + U^{(k)}(i+1, j) + U^{(k)}(i, j-1) + U^{(k)}(i, j+1) + f(i, j) \right),$$

$$\forall i, j = 2, \dots, n-1.$$

2. compute the convergence criterion as the norm of the increment between $U^{(k+1)}$ and $U^{(k)}$ as follows:

$$e = \sqrt{h \sum_{i,j} (U^{(k+1)}(i,j) - U^{(k)}(i,j))^2}.$$

If e is smaller than a prescribed tolerance, stop.

We have denoted by h the mesh spacing, assuming equal spacing in x and y , i.e. $h = 1/(n-1)$, while $f(i,j) = f(x_i, y_j)$.

First of all, even if you do not need to build the matrix explicitly, you can reinterpret the scheme as a Jacobi iteration for a linear system

$$A\mathbf{u} = \mathbf{f}$$

You have to number the nodes linearly, for instance by setting the node number associated with (i,j) to $k = ni + j$ (which corresponds to row ordering). In the linear system, $u_k = U(i,j)$ with $k = ni + j$.

2 The challenge

In this challenge, you are required to implement a parallel solver for the above problem for a generic f and solve the problem with $f(x) = 8\pi^2 \sin(2\pi x) \sin(2\pi y)$, whose exact solution is $u(x,y) = \sin(2\pi x) \sin(2\pi y)$.

In particular:

- The number of elements n , the forcing term f , and the number of parallel tasks should be specified by the user;
- The nodes are split into blocks of rows, as shown in the figure, and assigned to different MPI processes;
- The number of rows owned by each processor should be *balanced*, i.e., it should be as constant as possible among the different MPI ranks; see figure 1.

The tricky part is that the update formula requires access to values of U that are updated by other processors. When a row at the boundary between two MPI ranks has to be updated, data from adjacent entries must be properly exchanged.

For this challenge, you are required to implement a parallel C++ code in which:

1. Equation (1) is solved with the Jacobi iteration algorithm, by invoking appropriate MPI functions to communicate data between adjacent processors;
2. Appropriate **OpenMP** directives are included to further parallelize the local computations (hybrid parallelism);
3. Each processor checks its own local convergence criterion; then the local information is exchanged among all ranks. Convergence is reached if **all** ranks satisfy the stopping criterion;

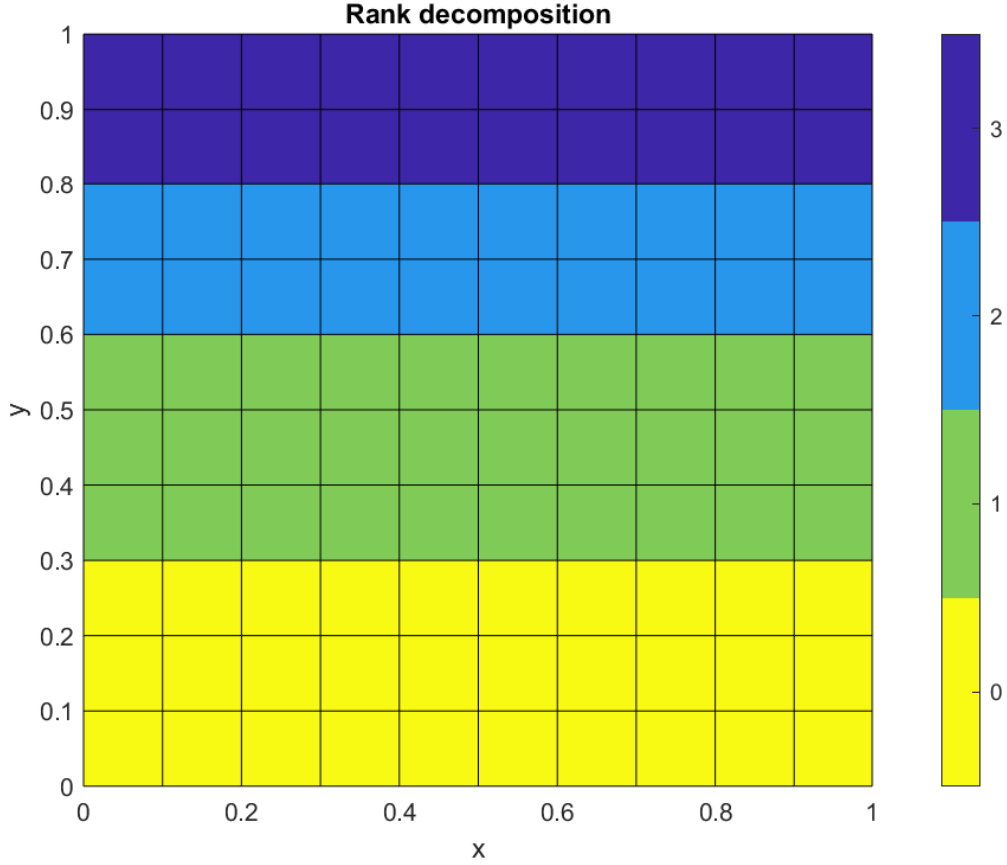


Figure 1: Example of a grid with $n = 11$ whose rows are distributed in parallel among 4 processes.

4. The solution and the grid coordinates are exported to one or more files for visualization. This task can be achieved, for example, by letting rank 0 collect all the local solutions and print them;
5. The solution is exported in `vtk` format so that it can be opened with ParaView or other tools able to read `vtk` files. The format can be found at this link.

Test and plot the performance of the code (serial vs. parallel) as the grid size increases ($n = 2^k$, $k = 4, \dots, 8$). Compare with the given exact solution in the L_2 norm

$$\sqrt{h \sum_{i,j} (f(i,j) - u(x_i, y_j))^2}.$$

and print the results as a function of n (or of h).

Remark: the Jacobi algorithm (without any preconditioner) converges for this problem, but at a very slow rate, particularly when n is large. Therefore, please choose tolerances wisely and set a large maximum number of iterations.

You are free to choose the code design and the data structures to use. The only mandatory requirement is that the algorithm should run **efficiently** in parallel!

Extras

- You may try to generalize the problem to one in which the boundary conditions are still of Dirichlet type, but are no longer homogeneous. This can be done by requiring the boundary conditions to be described as functions. This is a nice extension and is not particularly complicated. More complicated extensions are to allow boundary conditions of Neumann or Robin type: $\partial u / \partial n = h$ and $\partial u / \partial n + \alpha u = h$, respectively.
- Another possible extension is to implement a Schwarz-type iteration, which, in its simplest implementation, means that on each processor you do not perform Jacobi iterations, but instead solve the problem with a direct method (LU, for instance, by assembling the matrix) or with a more efficient iterative scheme, up to local convergence. The communication structure remains the same; only the local solver changes. The simplest Schwarz iteration is also called *block Jacobi*, since it is equivalent to performing the Jacobi iteration on the block matrix.

General rules

- We will use GitHub Classroom as in the previous challenge;
- The files you put in the Git repository should include header and source files, a Makefile, and a README file with a description, but **NOT** executables, binary libraries, or object files. These should be regenerated by the Makefile.
- If you refer to material in the **Examples/** directory of the course, use the environment variable `PACS_ROOT` to indicate it in the Makefile, in order to simplify things. Alternatively, copy the needed files from the Examples.
- You may use the Eigen matrix library if you wish.
- You may look at the Examples of the course for inspiration, but please follow your own ideas.
- Deliver a **test** folder. The **test** folder should contain a *script* that runs a small scalability test (with 1, 2, and 4 cores, for instance) and shows the performance of your code. Provide a **README.md** file with instructions to reproduce your results, and a **RESULT.md** file with a short discussion of what you have found.
- Inside the **test** folder, you may have a **data** folder where you collect some of your results. If you wish, you may use google benchmark or Catch2 for benchmarking.
- It may also be interesting to report your hardware in a file named **hw.info**. In Linux, `cat /proc/cpuinfo` should give the needed information, but you may also use other methods.

Evaluation criteria

The evaluation will be based on the following requirements:

- Use of GitHub;
- README file, comments in the code, and documentation;

- Quality of the code and its organization;
- Jacobi solver implementation;
- Scalability performance and discussion of the results;
- Quality and reproducibility of the results in the `test` folder;
- Hybrid MPI/OpenMP implementation;
- Extras:
 1. Set the Dirichlet boundary conditions also through user-defined functions and provide a test;
 2. Block Jacobi (one-level Schwarz).

A note: on a PC not designed for parallel computing, the results of a parallel computation may be very disappointing. You should get better results using the cluster to which you have access.